



7synchronization-II

Sıralı Tutarlılık ve Donanım Optimizasyonu

İşletim sistemlerinde ve donanımda işlemlerin, tüm işlemciler tarafından sanki kodda yazıldığı sırayla, arka arkaya (sekansiyel) çalıştırılıyormuş gibi görünmesine **Sıralı Tutarlılık (Sequential Consistency - SC)** denir.

Modern bilgisayarlarda sıralı tutarlılık yoktur. Donanımlarımız (CPU) ve derleyicilerimiz (Compiler) inanılmaz derecede "sabırsız"dır ve hızlanmak için sürekli **optimizasyon** yaparlar.

Örneğin, bellekten bir verinin gelmesini beklerken boş durmak yerine, kodun daha aşağısındaki bağımsız bir işlemi öne çekip çalıştırırlar. Buna **Kodun Yeniden Sıralanması (Code Reordering)** denir. Döngüleri birleştirirler, bazı işlemleri atlarlar veya önbellekteki (cache) verileri bekletirler.

Multitasking Sorunu ve Çözümü

Bu durum çoklu işlemlerde felaketle sonuçlanabilir. İşletim sistemleri, donanımın bu "başıboş hızlanma" huyunu kontrol altına almak için **Bellek Bariyerleri (Memory Barriers)** adı verilen mekanizmalar kullanır.

Kodun tam ortasına bir bariyer (çit) koyduğunda, CPU ve derleyiciye şu kesin emri verirsin: *"Dur! Bu bariyerden önceki tüm işlemler (belleğe yazmalar/okumalar) tamamen bitip onaylanmadan, bariyerin altındaki hiçbir işlemi çalıştırma ve sırasını değiştirme"*.

Mutex Mimarisi



Ortak veriyi korusun diye kullandığımız Mutex'in KENDİSİ bir ortak veridir ve o da Race Condition (Yarış Durumu) tehlikesi altındadır. Bu bölümde

mutex'in race condition'ı kendi içinde nasıl çözdüğünde göreceğiz.

İşletim sistemi önceki pdf'de öğrendiğimiz Mutex, semafor, monitör yapılarını senin kullanıcı tarafında rahatça kullanabilmen için *Pthread (POSIX Threads)* gibi kütüphanelerle sana sunar.

Pthread kütüphaneleriyle mutex yapılarının korunması ve mutex içinde race condition'ın engellenmesinden 2 temel yapı vardır:

1. Kesmeleri Kapatmak (Disabling Interrupts)

Bu yöntem en ilkel ve kaba kuvvete dayanan yöntemdir. İşlemciye dış dünyadan gelen uyarıları (klavye, fare, zamanlayıcı vb.) geçici olarak sağır etmektir.

- **Sonsuz Döngü Tehlikesi:** Sonsuz döngü gibi tehlikeli durumlarda CPU'nun kilitlenmesine neden olabilir.
- **Uyumak (Sleep) Sorunu:** Diyelim ki kapı kilitli ve beklemek için uykuya (**sleep**) dalacağız. Peki kesmeleri ne zaman geri açacağız? Uykuya dalmadan hemen önce açarsak, tam gözlerimizi kapatırken araya başka bir iş parçacığı girip koşulları değiştirebilir ve biz gelen "uyan" sinyalini kaçırıp sonsuza kadar uyuyabiliriz (Deadlock). Uyuduktan sonra açmak da imkansızdır, çünkü zaten uyuyoruz!
 - **Çözüm:** İşletim sistemleri, uykuya dalan bir thread'in ardından çalışmaya başlayan diğer thread'e veya bu thread uyandırıldığında ilk iş olarak kesmeleri açma görevini verir.
- **Akıllıca Kullanım:** Kesmeleri sadece mutex'in kendi iç durumunu değiştirdiğimiz kısa "meta-kritik" anlarda kapatmalıyız.
- **Çok Çekirdekli Çıkmazı:** Bu yöntem eski, tek çekirdekli sistemlerde harikaydı ancak çok çekirdekli yeni işlemcilerde bir çekirdekteki interrupt'ları kapatmak diğer çekirdeklerde thread'leri çalışmasını engellemez. Bütün çekirdeklere mesaj gönderip "Siz de durun" demek ise inanılmaz derecede yavaştır.

2. Spinlocks ve Donanım Desteği

Atomik Komutlar

Çok çekirdekli sistemlerde kesmeleri kapatamıyorsa ne yapacağız? Donanımın (CPU) kendisine başvuracağız! İşlemciler bize **Atomik (Bölünemez)** özel komutlar sunar. Bunların en meşhuru **Test-and-Set (Test Et ve Ayarla)** komutudur.

Bu komut, bir değişkenin değerini okur ve anında oraya "1" (Dolu) yazar. İşin büyüğü şundadır: Bu okuma ve yazma işlemi donanım seviyesinde tek bir paket halinde yapılır. İkisi arasında hiçbir işlem araya giremez.

Spinlock Nedir?

Thread kapıya gelir ve Test-and-Set ile kapıyı zorlar. Eğer kapı zaten 1 ise (kilitliyse), işletim sistemine gidip uyumak (`sleep`) yerine, kapının kolunu saniyede milyonlarca kez indirip kaldırarak "Açıldı mı? Açıldı mı? Açıldı mı?" diye sürekli dener. Yani olduğu yerde "spin" atar.

Ne Zaman İyidir?

Eğer içerideki iş çok kısaysa, uykuya dalıp sonra tekrar uyanmanın getireceği o ağır maliyetten (Context Switch) kaçınmak için kapıda dikilip beklemek çok daha hızlı ve mantıklıdır. Ayrıca çok çekirdekli sistemlerde sorunsuz çalışır.

Ne Zaman Kötüdür?

Kapıda dikilen iş parçacığı işlemciyi (CPU) boş yere yorar ve ısıtır (Busy-Waiting).

İşin daha kötüsü **Öncelik Tersinmesi (Priority Inversion)** dediğimiz felakettir (Mars Rover aracı bu yüzden çökmüştür): Düşük öncelikli bir thread içerideyken kilitliyse ve yüksek öncelikli bir thread gelip Spinlock ile kapıya dayanırsa ne olur? Yüksek öncelikli thread sürekli CPU'yu kullanıp kapıyı zorlar, CPU'yu bırakmadığı için de içerideki düşük öncelikli thread hiçbir zaman CPU bulup işini bitiremez ve kapıyı açamaz. Sonuç: Sistem sonsuza kadar donar!

Spinlock Performansını Attırma & Ticket Locks & Futex

Standart Spinlock'lar Neden Yetersizdir?

Basit bir "Test Et ve Ayarla" (Test-and-Set) Spinlock'unun iki devasa kusuru vardır:

- **Adaletsizlik:** Kapıda bekleyen 5 işlem (thread) varsa, içerideki kişi çıkıp kilidi açtığı anda kapıdaki 5 kişi aynı anda kapıya hücum eder. Kimin daha önce geldiğinin hiçbir önemi yoktur; şanslı olan (veya işlemci tarafından o an seçilen) içeri girer. İlk gelen, sonsuza kadar kapıda bekleyebilir.

- **Performans Kaybı:** Kapıda bekleyen herkes sürekli belleğe gidip "Kapı açıldı mı?" diye sorduğu için, sistemin donanım yolları (bus) tıkanır. Bellek ile işlemciler arasındaki bu sürekli ve boş laf kalabalığı, sistemi inanılmaz yavaşlatır.

Ticket Locks - Fırın Sırası Mantiği

Adaletsizliği çözmek için bankalardaki veya fırınlardaki "Sıramatik" sistemini icat etmişler. İşletim sistemi donanımdan yeni bir atomik komut ister: **Al ve Artır (Fetch-and-Add)**.

- Kapıya gelen her işlem, sıramatikten bir bilet (`ticket`) alır ve makinedeki numara bir artar.
- Kapının üzerinde de yanan bir "Sıra Kimde" (`turn`) tabelası vardır.
- Her işlem kendi elindeki bilet ile tabeladaki sayıyı karşılaştırır. Eşit değilse kapıda döner (spin atar). İçerideki işlem çıkarken tabeladaki sayıyı 1 artırır.
- **Sonuç:** Adalet (Fairness) %100 sağlandı! Artık kimse sırasını başkasına kaptırmıyor. Ama **performans sorunu hala devam ediyor**, çünkü sırası gelmeyen 10 kişi hala ellerindeki fişle saniyede milyonlarca kez tabelaya bakıp duruyor (işlemciyi sömürüyor).

Pes Edip Sırayı Salmak (Yielding)

Madem sıram gelmedi, boş boş ekrana bakıp CPU'yu yormayayım diyen işlemler için `yield()` (devret) komutu eklenir. Bir işlem bakar ki sıra onda değil, anında işletim sistemine *"Benim işlemci hakkımı başkasına ver, ben kenara çekiliyorum"* der.

Neden tam çözüm değil? Bu CPU'yu rahatlatır ama, diyelim ki kapıda 100 kişi bekliyor. İşletim sistemi sırayla 100 kişiye CPU'yu verir, hepsi "Sıra bende değilmiş" deyip geri bırakır. Bu sürekli "al-bırak" (context switch) işlemi kendi başına devasa bir zaman kaybıdır.

Sleeping Locks & Guard

Boş yere dönmek (Spin) yerine bekleyenleri uyutmalıyız! Yeni tasarımda, kilit verisinin içine bir "Uyuyanlar Kuyruğu" eklenir.

- Sadece o kuyruğa ismimizi yazdıracağımız ve "kapı dolu mu" diye bakacağımız o milisaniyelik kısım için minik bir Spinlock (`guard`) kullanılır.
- Eğer kapı doluysa, işlemciyi meşgul etmez, doğrudan uyku kuyruğuna girip derin bir uykuya dalarız. İçeriden çıkan kişi, çıkarken kuyruğa bakar ve uyuyan birini uyandırır. Çok daha verimlidir.

Linux Futex

Uyutmak harika bir çözümdür ama birini uyutmak veya uyandırmak için her seferinde İşletim Sisteminin derinliklerine (Kernel'a) inip çıkmak (System Call) çok ama çok yavaştır. Linux mühendisleri şu dahiyane soruyu sordu: *"Eğer kapı zaten boşsa ve kimse beklemiyorsa, neden boş yere Kernel'a inip zaman kaybedelim?"*

İşte **Futex** böyle doğdu. Futex'in kalbinde kilidin 3 durumu vardır:

- **0 (UNLOCKED):** Kapı boş.
- **1 (LOCKED):** İçeride biri var ama kapıda bekleyen (uyuyan) KİMSE YOK.
- **2 (CONTESTED):** İçeride biri var ve kapıda uyuyan EN AZ BİR KİŞİ VAR.

Futex'in Çalışma Mantığı (Karşılaştır ve Değiştir - Compare & Swap):

- **Rekabetsiz (Hızlı) Senaryo:** Kapiya geldin, durum `0`. Hiç Kernel'a haber vermeden durumu anında `1` yapar ve içeri girersin. Çıkarken de durumu `0` yapar çıkarsın. İşletim sisteminin ruhu bile duymaz! İnanılmaz hızlıdır.
- **Rekabetli (Yavaş) Senaryo:** Kapiya geldin ama durum `1` (İçeride biri var). Hemen durumu `2` yaparsın ve Futex ile Kernel'a gidip *"Ben uyuyorum, kapı açılınca beni dürt"* dersin.
- **Çıkış Senaryosu:** İçerideki kişi işini bitirip çıkarken kapıdaki sayıya bakar. Eğer `1` yazıyorsa, kapıda kimsenin olmadığını bilir ve sessizce `0` yapıp çeker gider (yine Kernel'a inmez). Ama kapıda `2` yazdığını görürse, *"Aha, ben içerideyken birileri gelip uyumuş"* der ve Kernel'a gidip onları uyandırır.

Özetle Futex; sadece ve sadece ortada bir tıkanıklık (uyuyanlar) varsa işletim sistemini rahatsız eden, onun dışında kendi işini kendi halleden mükemmel ötesi melez bir kilittir.

Donanım ve Bellek Mimarisi Katmanı

Kilitler (Mutex vb.) ne kadar optimize edilirse edilsin sistemi yavaşlatır. Acaba bu kilitlerden tamamen kurtulabilir miyiz?

Önbellek Tutarlılığı

Her CPU çekirdeğinin kendi içinde süper hızlı bir özel not defteri (L1/L2 Cache) ve bir de hepsinin ortak kullandığı devasa bir ana tahta (ram) vardır.

- Diyelim ki CPU-1, "A değişkenini 5 yap" dedi ve bunu sadece kendi not defterine yazdı. CPU-2 gidip ana tahtaya bakarsa eski değeri görecektir.
- İşte herkesin aynı gerçeği görmesi için donanım seviyesinde **MESI Protokolü** adı verilen bir fısıltı gazetesi (Bus protokolü) çalışır.
- **MESI Mantığı:** Bir değişkenin 4 durumu olabilir. Eğer sen o değişkeni değiştireceksen (Modified), diğer tüm işlemcilerle "Ben bunu değiştiriyorum, sizin defterlerinizdekini çizin (Invalid)!" diye bağırarak zorundasın.

NUMA Mimarisi ve MCS Kilidi

Eski Spinlock mantığında herkes aynı kapının kolunu zorluyordu (Test-and-Set) ve bu da işlemciler arası ağda (Bus) korkunç bir trafiğe sebep oluyordu.

TTAS (Test-and-Test-and-Set): İşlemciler önce "Kendi önbelleğimdeki değere bakayım, değiştiğinde gidip kapıyı zorlayayım" diyerek trafiği biraz azalttılar.

MCS Kilidi (Kuyruk Mantığı): Ama asıl devrim budur. Herkesin tek bir kapıya yığılması yerine, bekleyen işlemlerden bir **kuyruk (Linked List)** oluşturulur. Her işlem, sadece kendi önündeki işlemin omuzuna dokunur ve "Kendi işin bitince bana haber ver" diyerek **sadece kendi yerel değişkeni üzerinde** spin atar (bekler). Önündeki kişi çıkarken arkasındakinin değişkenini 1 yapar. Böylece ağda hiçbir fırtına kopmaz, sırası gelen sessizce içeri girer.

Bellek Bariyeri ve Release-Acquire İkili

Bir önceki bölümde derleyicilerin kodların sırasını değiştirdiğinden bahsetmiştik. Kilitli programlamada işleri sıraya sokmak için görünmez duvarlar (Bariyerler) kullanırız:

- **Release (Bırakma/Yazma) Bariyeri:** Bu duvarın üstündeki hiçbir işlem, duvarın altına inemez. "Benim hazırlıklarım bitmeden, sonucu kimseye gösterme"

demektir.

- **Acquire (Alma/Okuma) Bariyeri:** Bu duvarın altındaki hiçbir işlem, duvarın üstüne çıkamaz. "Veriyi tam olarak teslim almadan, alt satırlardaki hiçbir okumayı başlatma" demektir.

Bu iki duvarı eşleştirdiğinde, farklı işlemciler kilit (Mutex) kullanmadan birbirleriyle kusursuz bir sırayla haberleşebilir.

Kilitsiz (Lock-Free) Veri Yapıları ve ABA Problemi

Eğer donanımın bize sunduğu "Karşılaştır ve Değiştir" (Compare-And-Swap - CAS) komutunu kullanırsak, yığıt (Stack) gibi yapılarda hiç Mutex kullanmadan işlem yapabiliriz. Komut basittir: *"Eğer en üstteki eleman hala benim beklediğim elemansa, yenisini onun üzerine koy."*

1. Sen en üstte "A" elemanını görürsün ve yeni eleman eklemek için hazırlık yaparken işletim sistemi seni uyutur.
2. O sırada başka bir işlem gelir. A'yı çıkarır, altındaki B'yi çıkarır, sonra işi bitip **A'yı tekrar en üste geri koyar.**
3. Sen uyanırsın. CAS komutuyla bakarsın: *"Aaa, en üstte hala A var, demek ki ben uyurken kimse buraya dokunmamış!"* dersin ve işlemini yaparsın.
4. **Büyük Patlama:** Halbuki alt taraftaki bütün yapı (B silinmişti vs.) değişmiştir ama sen sadece en üste baktığın için bunu fark edemezsin ve yığıtın bağlantılarını tamamen koparıp sistemi çökertirsin!

Çözüm için elemanlara versiyon numarası eklemek veya tehlike işaretçileri (hazard pointers) kullanmak gerekir.

RCU (Read-Copy-Update) Algoritması

Özellikle ağ yönetimindeki devasa yönlendirme tabloları (routing tables) saniyede milyonlarca kez okunur ama çok nadir güncellenir. Eğer buraya klasik bir kilit koysaydık, internet çökerdi. Linux çekirdeğinin yıldızı **RCU**'nun mantığı Wikipedia gibidir.:

- Milyonlarca kiři aynı anda makaleyi okur (Hiç kilit yoktur, hiç beklemezler).
- Bir görevli makaleyi güncelleyeceğinde, makalenin **bir kopyasını (Copy)** alır. Kopyanın üzerinde çalışır (**Update**).
- İşini bitirince, sitenin ana bağlantısını tek bir hamleyle yeni makaleye yönlendirir.
- Eski makaleyi okuyanlar okumaya devam eder. Ne zaman ki eski makaleyi okuyan **son kiři** de sayfadan çıkar, işte o zaman eski makale bellekten silinir. Mükemmel bir performans sanatıdır!

Donanımsal İşlemsel Bellek (Transactional Memory - TSX)

Modern işlemciler (örneğin Intel TSX), "Acaba kilit kullanmasam ne olur?" diyerek kodları spekülatif (tahmini) olarak çalıştırır. Kilit varmış gibi davranır ama aslında kilitlemez. Eğer arka planda gerçekten kimse o veriye dokunmamışsa, işlemi bir çırpıda yazar geçer (Lock Elision - Kilidi Yok Sayma). Ama iki işlemcinin aynı veriye dokunduğunu fark ederse (Conflict), bir işlemi derhal iptal eder (Abort), başa sarar ve bu sefer paşa paşa gerçek kilidi kullanarak içeri girer.

Öğrendiğimiz Tüm Mimarilerin Katmanları ve Çözdükleri Problemler (Özet)

1. Yazılım / Kullanıcı Katmanı

- **Araçlar:** Pthread, Mutex, Semafor, Monitör, Durum Değişkenleri.
- **Neyi Çözer?** Senin yazdığın C, Java veya Python programlarında, birden fazla iş parçacığının aynı değişkene (örneğin bir banka hesabına) aynı anda ulaşip veriyi bozmasını engeller.
- **Özet:** Binaların dış cephesidir. Biz programcılar sadece burayı görür ve kullanırız. Arabanın direksiyonudur.

2. İşletim Sistemi Çekirdeği Katmanı

- **Araçlar:** Kesmeleri Kapatma (Interrupt Disabling), Standart Spinlock'lar, Futex.
- **Neyi Çözer?** 1. katmandaki o Mutex'lerin ve Semaforların arka planda *gerçekten* çalışabilmesini sağlar. İşletim sistemi, "Bir thread Mutex'te beklerken işlemciyi boşuna yormasın, onu uyutayım (Sleep), kapı açılınca uyandırayım (Wake)" mantığını burada, donanıma emirler göndererek çözer.
- **Özet:** Kaputun altındaki motordur.

3. Donanım ve Bellek Mimarisi Katmanı (Hardware Level)

- **Araçlar:** Atomik Komutlar (Test-and-Set, Compare-and-Swap), Önbellek Tutarlılığı (Cache Coherence - MESI Protokolü).
- **Neyi Çözer?** Her işlemci çekirdeğinin kendine ait süper hızlı bir önbelleği (L1/L2 Cache) vardır. Eğer Çekirdek-1 bir değişkeni değiştirirse, Çekirdek-2'nin bundan nasıl haberi olacak? İşte donanım seviyesindeki **MESI protokolü**, çekirdeklerin anakart üzerindeki yolları (Bus) kullanarak birbirlerine "Ben bu veriyi değiştirdim, sen kendi önbelleğindeki sil" diye bağırmasını sağlar.
- **Özet:** İşlemcinin içindeki fiziksel kablolar ve elektronik sinyalleşmedir. İşletim sistemi bile buraya doğrudan müdahale edemez, donanım bunu kendi içinde halleder.

4. Kilitlerin Tamamen Çöpe Atılması

- **Araçlar:** Lock-Free (Kilitsiz) Veri Yapıları, RCU (Read-Copy-Update), Donanımsal İşlemsel Bellek (Transactional Memory).
- **Neyi Çözer?** Mühendisler baktılar ki, 3. katmandaki o çekirdeklerin birbirine sürekli "Kapı açıldı mı? Veri değişti mi?" diye bağırması sistemi inanılmaz yavaşlatıyor. Dediler ki: *"Madem kilitler (Mutex/Spinlock) donanımı bu kadar yoruyor, kilit kullanmadan kod yazalım!"*
 - **Lock-Free Stacks:** Sadece donanımın "Karşılaştır ve Değiştir" (CAS) komutunu kullanarak, hiç kilit vurmada yığıta (Stack) eleman ekleme sanatı.

- **RCU:** Kilit vurmak yerine verinin kopyasını alıp, güncelleyip, eskisini çöpe atma sanatı. Okuyanlar asla ama asla beklemez.
- **Özet:** Burası modern işletim sistemleri mühendisliğinin sınır uçlarıdır. Performansın tanrısıdır.

Toparlarsak

1. Biz önce programcı olarak kilitleri (Mutex) öğrendik (Katman 1).
2. Sonra işletim sisteminin bunu nasıl yaptığını gördük (Katman 2).
3. Sonra çok çekirdekli donanımların bu kilitler yüzünden nasıl tıkanıldığını ve MESI ile nasıl haberleştiğini inceledik (Katman 3).
4. En sonunda da "Lanet olsun kilitlere!" deyip kilitsiz (Lock-Free/RCU) yapıların icadına şahit olduk (Katman 4).